Constraint Satisfaction

Fundamental Topics · CSP · Backtracking · Constraint Propagation

Course No: 0613-401

Course Title: Artificial Intelligence

Ekramul Haque Amin

Lecturer

Department of Artificial Intelligence and Data Science

Dhaka International University (DIU)

Today's Content

- **Part 1 — Important fundamental topics**
  - Four approaches to AI; foundations of AI; AI problems
  - Defining a problem as state-space search: the water jug problem
  - Problem characteristics
  - Search with partial information
- **Part 2 — Constraint Satisfaction Problems (CSP)**
  - CSP formulation: map colouring of Bangladesh
  - Backtracking search — step by step
  - Constraint propagation: forward checking, arc consistency
  - N-Queens with backtracking; Python code

Part 1

Some important fundamental topics we have not covered yet

Approaches to AI · Foundations · AI problems · State space · Problem characteristics · Partial information

Four Approaches to AI

	Like humans	Rationally (the right thing)
THINKING	**Thinking humanly** Cognitive modelling: build programs that think the way people think (cognitive science).	**Thinking rationally** "Laws of thought": correct reasoning with logic. E.g. All men are mortal; Socrates is a man → Socrates is mortal.
ACTING	**Acting humanly** The Turing Test: behave so that a judge cannot tell machine from human (NLP, knowledge, reasoning, learning).	**Acting rationally ★** The rational agent: act to achieve the best expected outcome. This course's approach.

Comparing the Approaches

Approach	Goal	Success is measured by…	Example
Turing Test (acting humanly)	Behave like a human	A judge cannot tell it from a person	A chatbot in a text conversation
Human-like thinking	Think like a human	Its reasoning steps match human thinking	A model of how students solve algebra problems
Laws of thought	Reason correctly	Logically valid conclusions	A theorem prover
Rational agent	Do the right thing	Best expected performance measure	A self-driving car maximising safety and speed

- **The rational agent approach is the most general and the easiest to test, so modern AI (and this course) uses it.**

Foundations of AI

Discipline	What it gave AI
Philosophy	Logic, reasoning, mind, how knowledge leads to action
Mathematics	Formal logic, computation (algorithms), probability
Economics	Decision theory and utility: making the best choice
Neuroscience	How the brain processes information → neural networks
Psychology	How humans think and act → cognitive science
Computer engineering	Fast computers and hardware to run AI programs
Control theory	Feedback and self-regulating systems
Linguistics	Language structure and meaning → natural language processing

AI Problems: Task Domains

Mundane tasks	Formal tasks	Expert tasks
Perception: vision, speech	Games: chess, checkers	Engineering: design, fault finding
Natural language understanding	Mathematics: geometry, logic	Scientific analysis
Common-sense reasoning	Proving theorems	Medical diagnosis
Robot control		Financial analysis

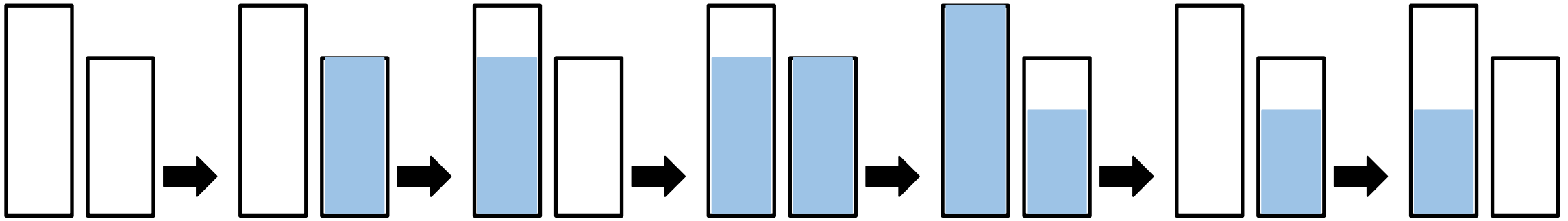
- Easy for people ≠ easy for computers: mundane tasks are often the hardest.
- Expert tasks need a lot of specialised knowledge → expert systems (later in the course).

State-Space Search: The Water Jug Problem

- **Jugs: 4 litres and 3 litres, no markings; a tap to fill them.**
- **Goal: exactly 2 litres in the 4-litre jug.**
- **State: (x, y) = water in the 4-L and 3-L jug. Start (0, 0); goal (2, any).**

Rule	Condition	New state
Fill 4-L jug	$x < 4$	(4, y)
Fill 3-L jug	$y < 3$	(x, 3)
Empty 4-L jug	$x > 0$	(0, y)
Empty 3-L jug	$y > 0$	(x, 0)
Pour 3-L → 4-L until 4-L is full	$x + y \geq 4$	$(4, y - (4 - x))$
Pour all of 3-L into 4-L	$x + y \leq 4$	(x + y, 0)
Pour 4-L → 3-L until 3-L is full	$x + y \geq 3$	$(x - (3 - y), 3)$
Pour all of 4-L into 3-L	$x + y \leq 3$	(0, x + y)

Water Jug: One Solution, Step by Step



- Left jug = 4 litres, right jug = 3 litres; blue = water.
- The solution is a path of states: 6 moves from (0, 0) to (2, 0).

Problem Characteristics (1/2)

Question	Yes — example	No — example
1. Can the problem be broken into smaller independent parts?	Integration: $\int(x^2 + 3x)\,dx = \int x^2\,dx + \int 3x\,dx$	Blocks world: moving one block affects others
2. Can solution steps be ignored or undone?	8-puzzle: a move can be undone (recoverable)	Chess: a move cannot be taken back (irrecoverable)
3. Is the outcome of each action predictable?	8-puzzle: we know exactly what a move does	Card games / bridge: hidden cards, uncertain
4. Is ANY solution good enough, or only the BEST?	“Is there a path?” — any path is fine	Travelling salesman: only the shortest route

Problem Characteristics (2/2)

Question	Example A	Example B
5. Is the solution a STATE or a PATH?	State: 8-queens — only the final board matters	Path: water jug — we need the sequence of moves
6. How much knowledge is needed?	Little: chess needs only the rules (plus search)	A lot: understanding a newspaper story
7. Does it need interaction with a person?	No: a theorem prover works alone	Yes: a medical system asks the patient questions

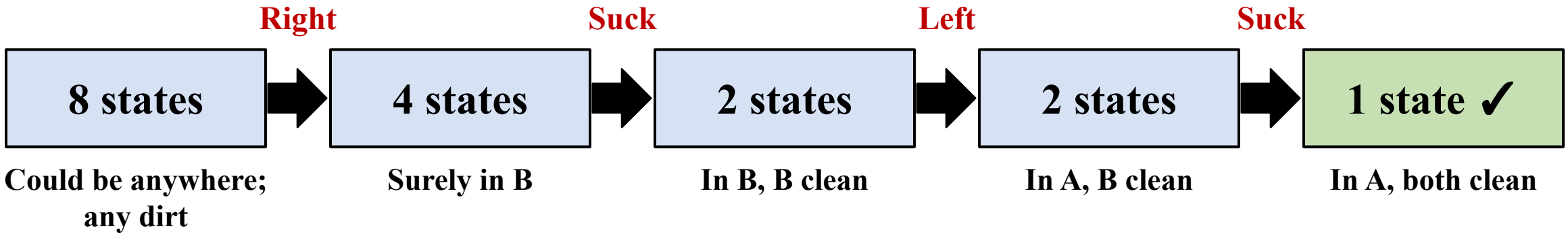
- Exercise: answer all 7 questions for the 8-puzzle (answers in notes).

Search with Partial Information

Problem type	What the agent lacks	What it does instead
Sensorless (conformant)	No sensors: does not know which state it is in	Searches over BELIEF STATES (sets of possible states)
Contingency	Partial observation or uncertain actions	Builds a plan with IF–THEN branches; senses while acting
Exploration	Does not know the states or actions at all	Explores the world to learn it (online search)

- Link to intelligent agents: these are the partially observable and stochastic environments.

Example: A Sensorless Vacuum Agent



- **Belief state = the set of states the agent might be in.**
- **The plan [Right, Suck, Left, Suck] reaches the goal from every possible start.**
- **With sensors (contingency), the agent could instead check for dirt and choose actions with IF–THEN.**

Part 2

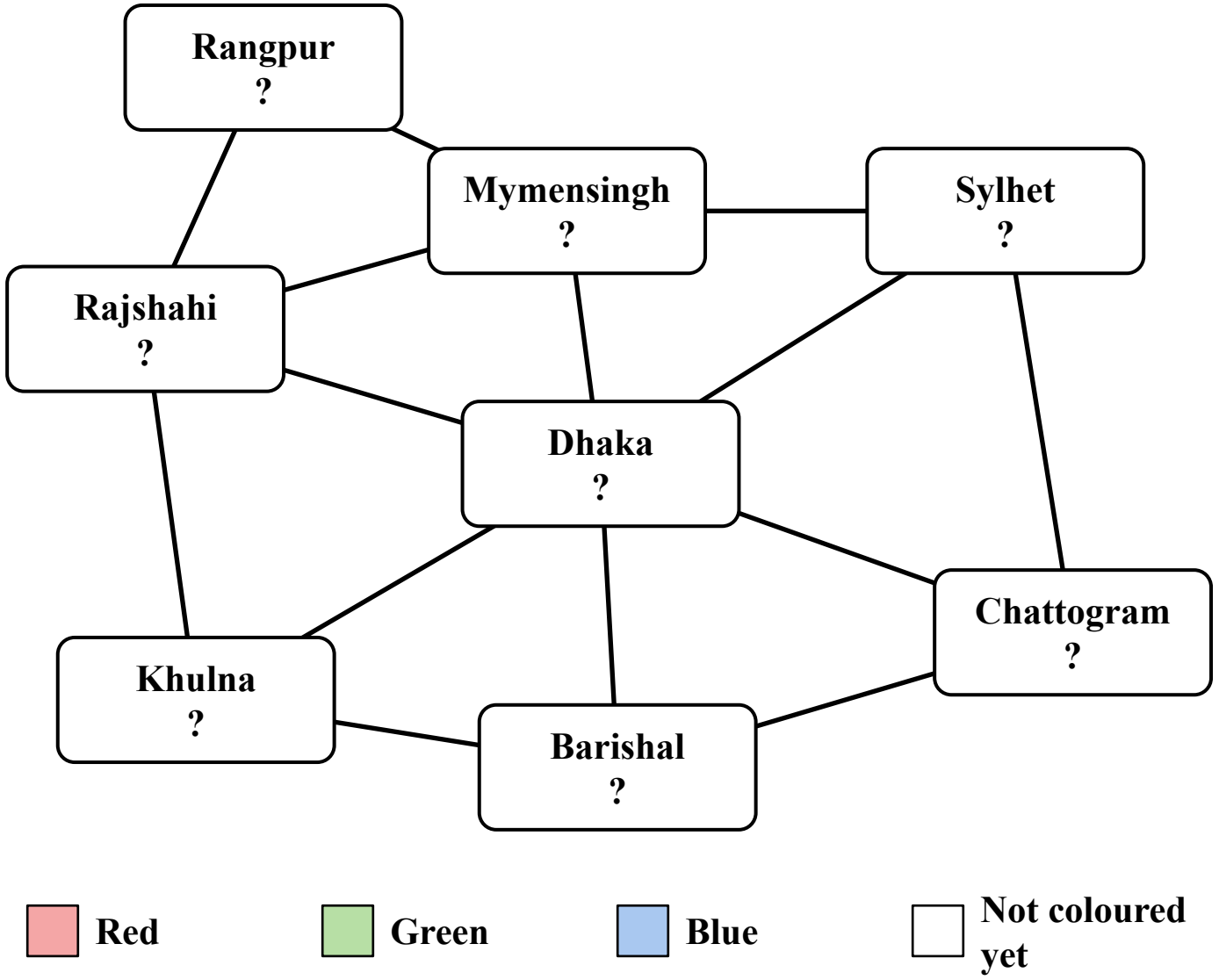
Constraint Satisfaction Problems (CSP)

Formulation · Backtracking · Constraint propagation

What Is a CSP?

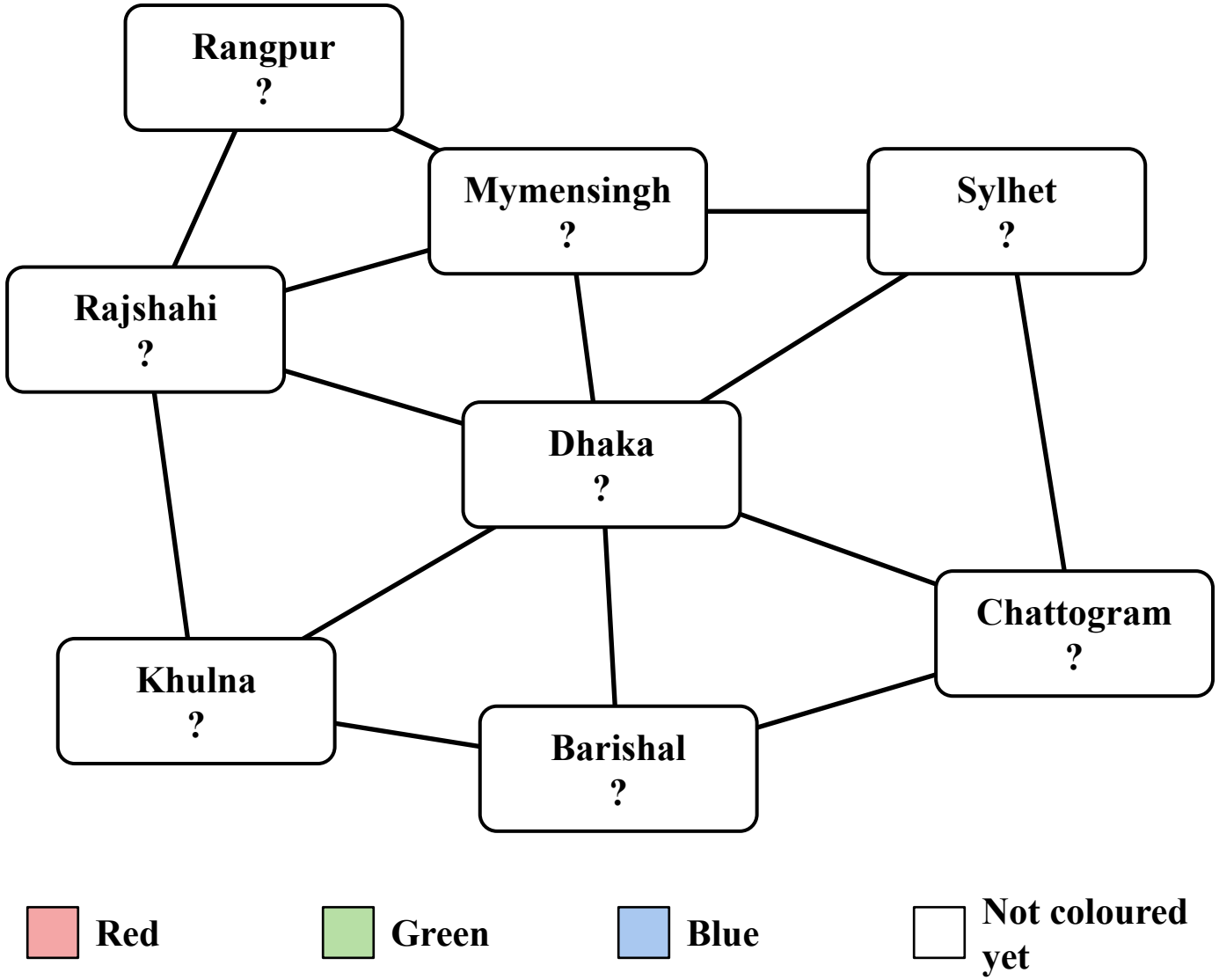
- **A CSP has three parts:**
  - **Variables: X1, X2, …, Xn**
  - **Domains: the allowed values for each variable**
  - **Constraints: rules about which combinations are allowed**
- **A solution = a value for every variable that satisfies all constraints.**
- **Only the final assignment matters, not the path (a STATE problem).**
- **Everyday examples: class timetables, exam scheduling, Sudoku, map colouring, N-Queens.**

Example: Colouring the Divisions of Bangladesh



- **Variables:** the 8 divisions
- **Domain:** {Red, Green, Blue}
- **Constraints:** neighbours ≠ same colour (14 borders, shown as lines)
- e.g. Dhaka ≠ Sylhet, Khulna ≠ Barishal
- Goal: colour all 8

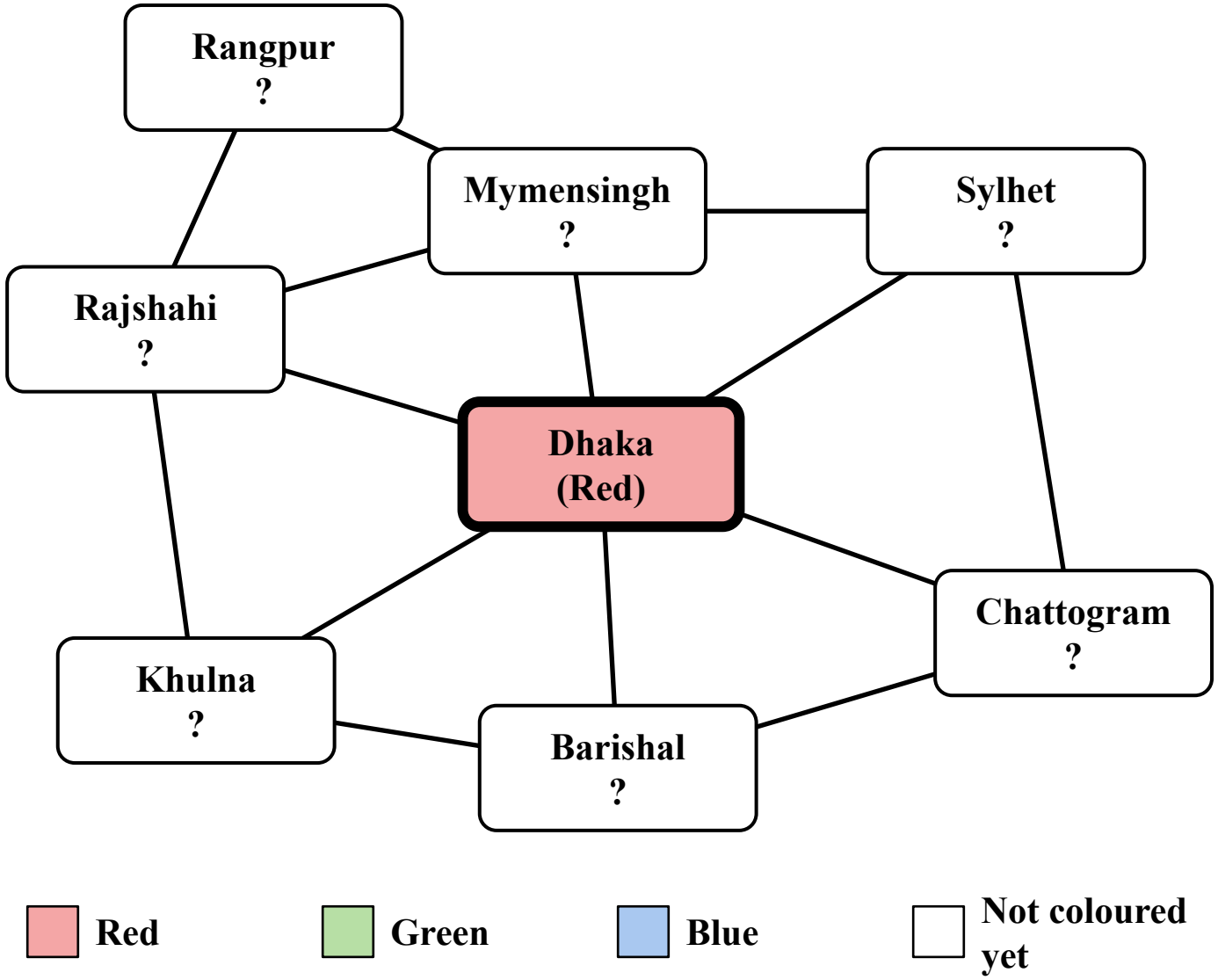
Backtracking Search: Step by Step (0/7)



Backtracking: the idea

- Assign variables one at a time.
- Pick the first colour that does not clash with coloured neighbours.
- If a variable has no legal colour → go back (backtrack) and change an earlier choice.

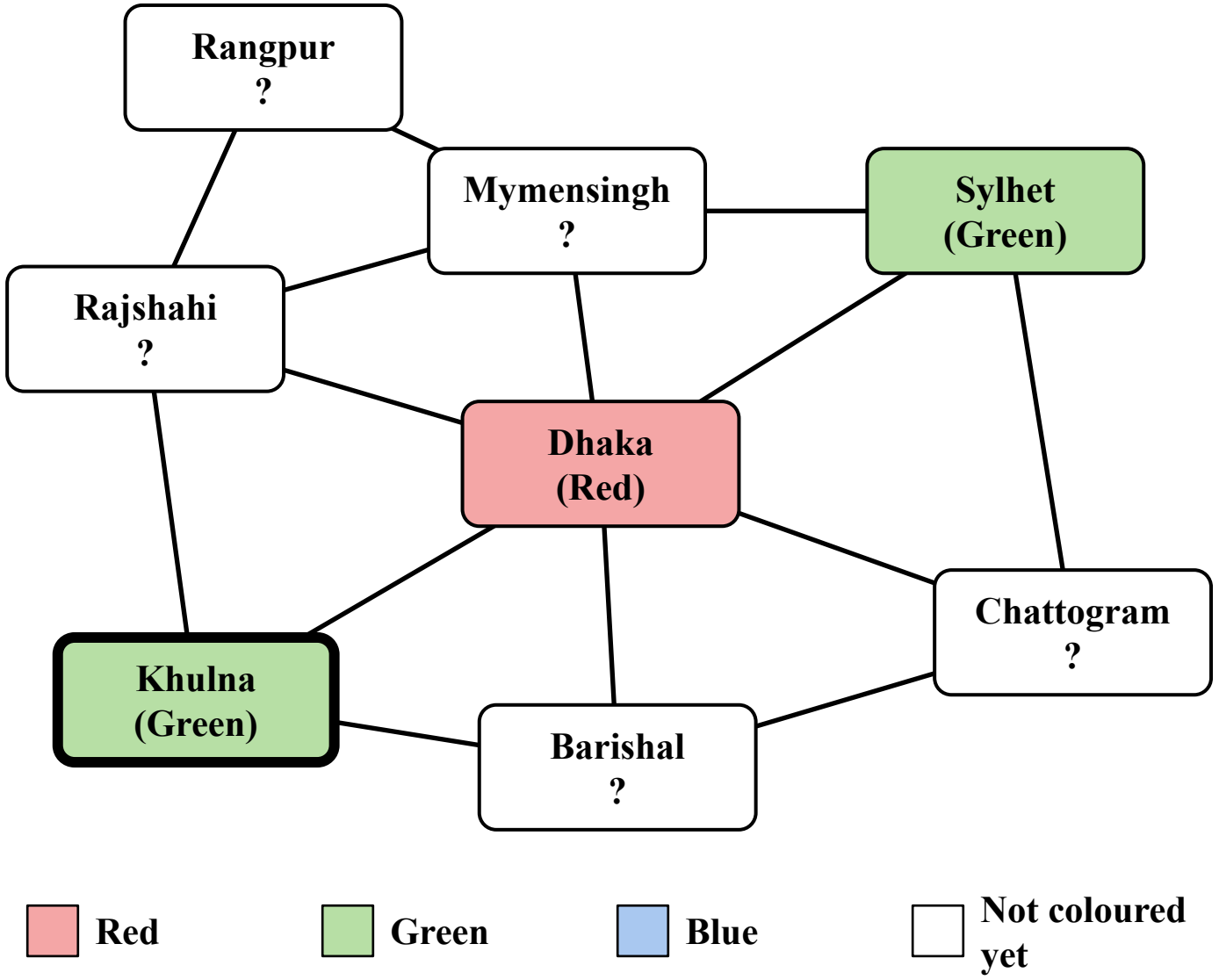
Backtracking Search: Step by Step (1/7)



Step 1: Dhaka = Red

- Order: Dhaka, Sylhet, Khulna, Rajshahi, Mymensingh, Chattogram, Barishal, Rangpur
- Try colours in order Red, Green, Blue.

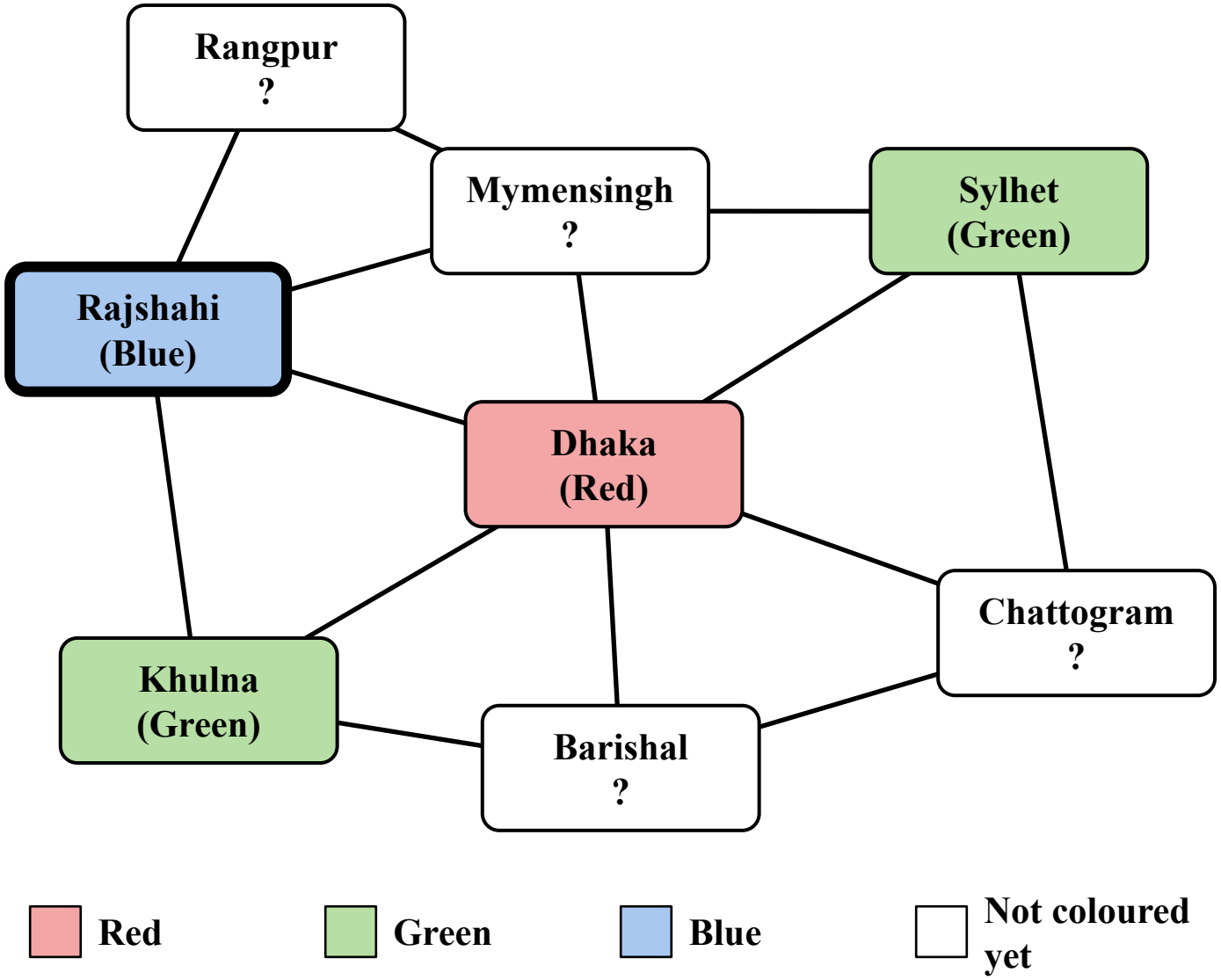
Backtracking Search: Step by Step (2/7)



Steps 2–3: Sylhet, Khulna

- Sylhet: Red ✗ (Dhaka) → Green
- Khulna: Red ✗ (Dhaka) → Green
  (Khulna is not next to Sylhet)

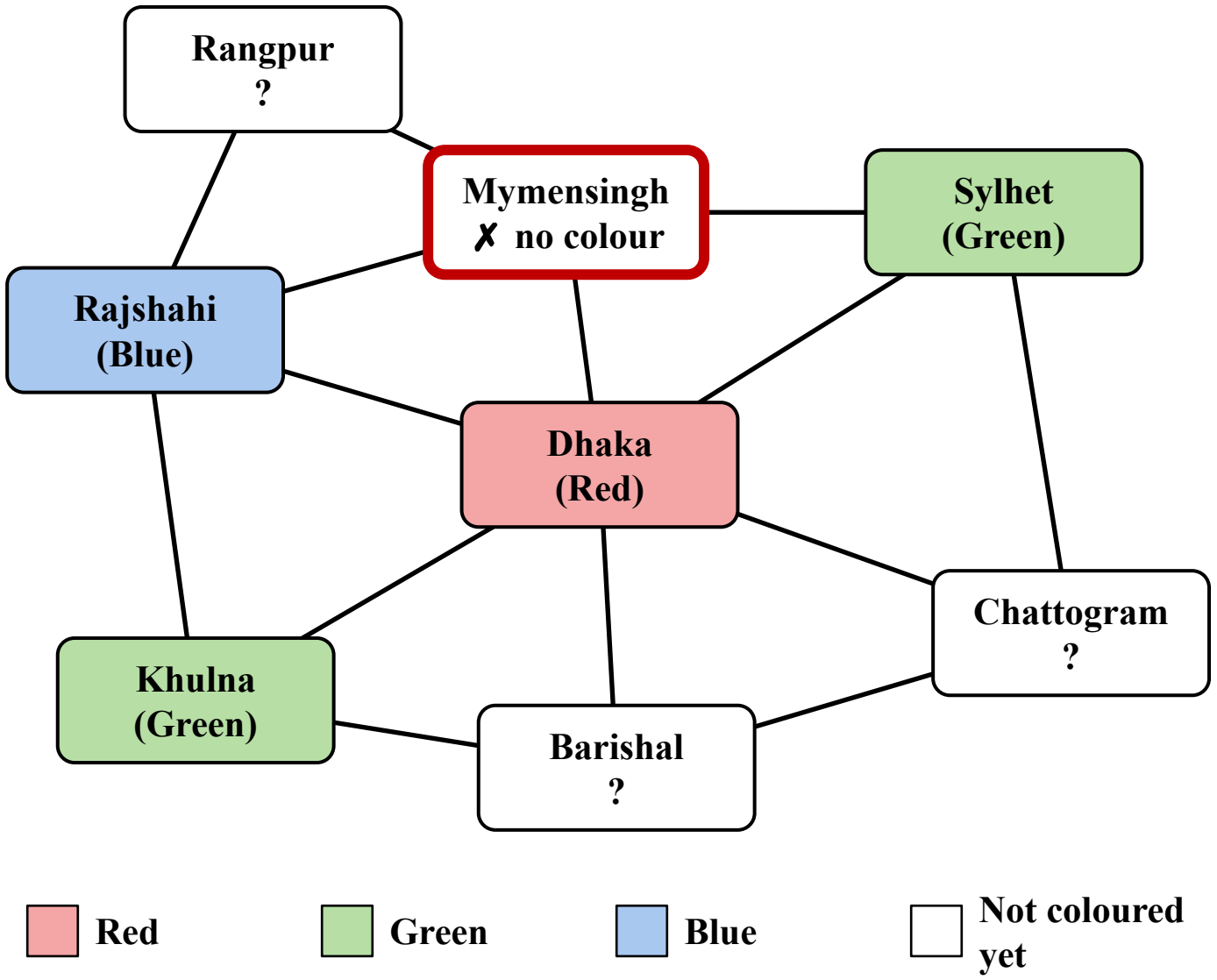
Backtracking Search: Step by Step (3/7)



Step 4: Rajshahi = Blue

- Red ✗ (Dhaka), Green ✗ (Khulna) → <u>Blue</u>

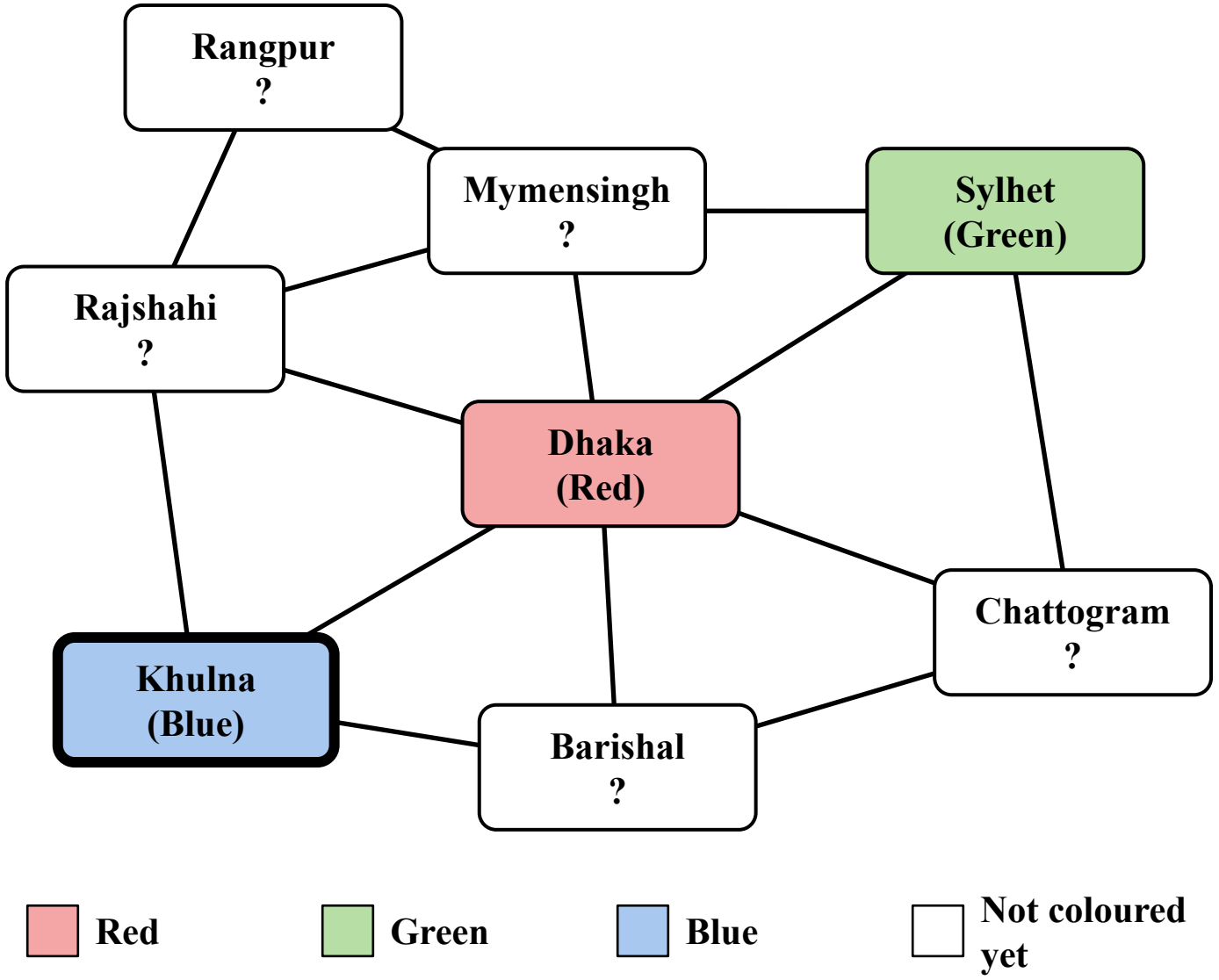
Backtracking Search: Step by Step (4/7)



Step 5: Mymensingh — DEAD END ✗

- Red ✗ (Dhaka)
- Green ✗ (Sylhet)
- Blue ✗ (Rajshahi)
- No colour left → backtrack!

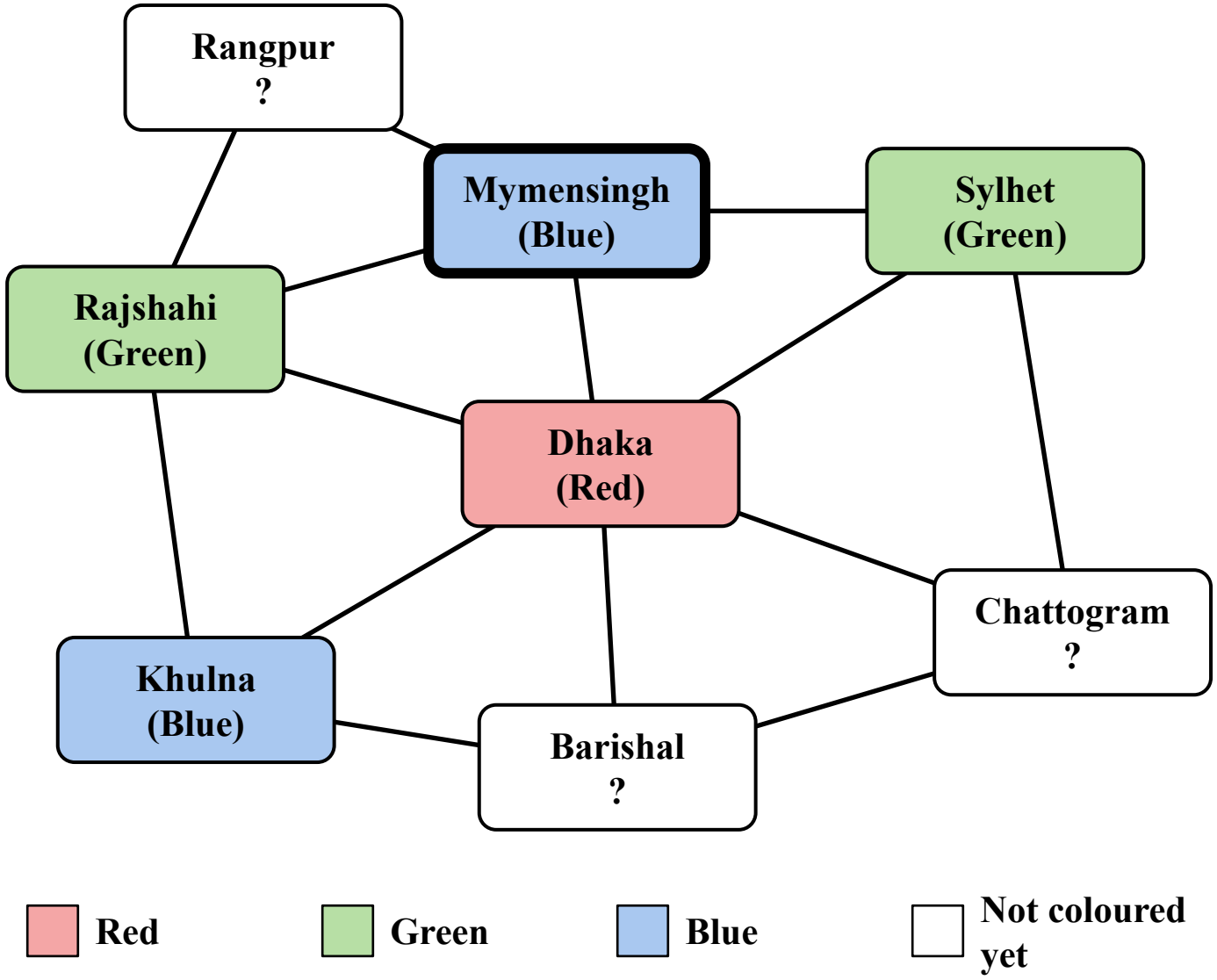
Backtracking Search: Step by Step (5/7)



Step 6: backtrack to Khulna

- Rajshahi has no other colour (Green ✗ because of Khulna) → undo it.
- Go back to Khulna and try its next colour: Blue.

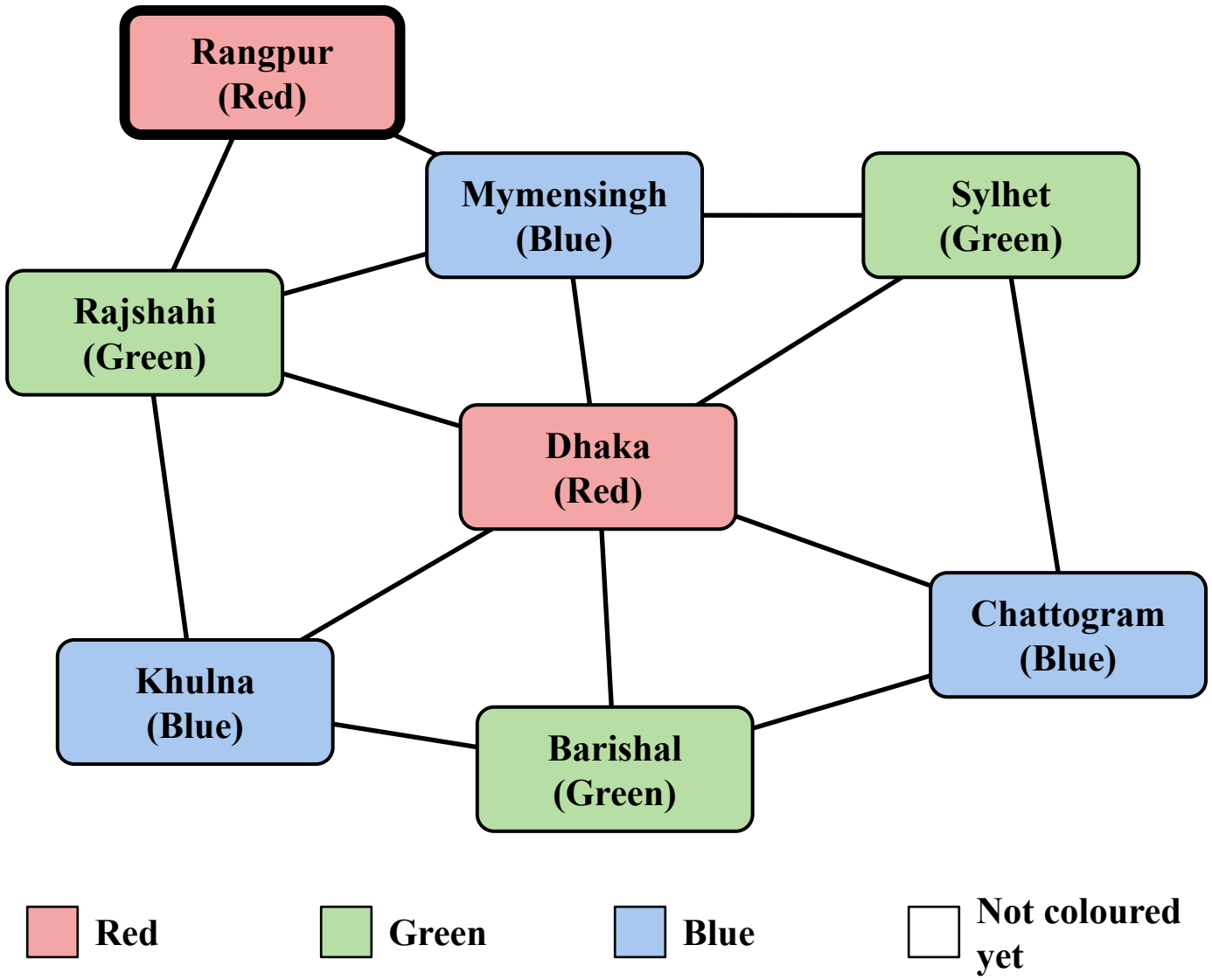
Backtracking Search: Step by Step (6/7)



Steps 7–8: Rajshahi, Mymensingh

- Rajshahi: Red ✗, <u>Green</u> ✓
- Mymensingh: Red ✗, Green ✗
  (Sylhet, Rajshahi) → <u>Blue</u> ✓

Backtracking Search: Step by Step (7/7)



Steps 9–11: SOLVED ✓

- Chattogram: Blue (≠ Dhaka, Sylhet)
- Barishal: Green (≠ Dhaka, Chattogram, Khulna)
- Rangpur: Red (≠ Rajshahi, Mymensingh)
- All 14 constraints satisfied, with 1 backtrack.

Backtracking Algorithm

```
function BACKTRACK(assignment):
   if every variable is assigned: return assignment
   var ← choose an unassigned variable
   for each value in the domain of var:
      if value does not clash with assigned neighbours:
         add var = value to assignment
         result ← BACKTRACK(assignment)
         if result ≠ failure: return result
         remove var = value      // backtrack
   return failure
```

- It is DFS on partial assignments; memory O(n) for n variables.

Constraint Propagation: Detecting Failure Earlier

After…	Dhaka	Sylhet	Khulna	Rajshahi	Mymensingh	Chattogram	Barishal	Rangpur
Start	R G B	R G B	R G B	R G B	R G B	R G B	R G B	R G B
Dhaka = Red	= Red	G B	G B	G B	G B	G B	G B	R G B
Sylhet = Green	= Red	= Green	G B	G B	B	B	G B	R G B
Khulna = Green	= Red	= Green	= Green	B	B	B	B	R G B
Rajshahi = Blue	= Red	= Green	= Green	= Blue	{ } empty ✗	B	B	R G

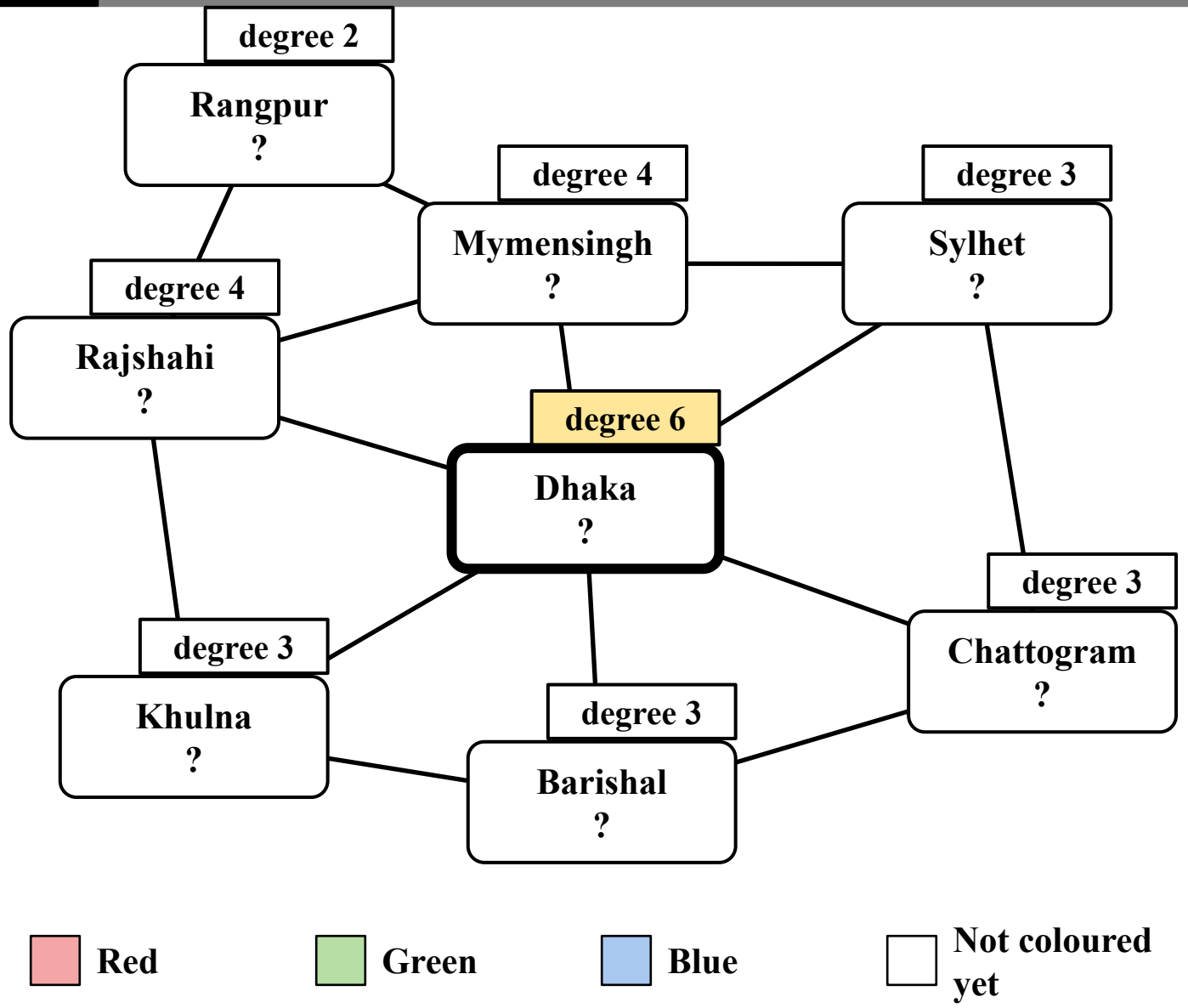
- **Forward checking:** after each assignment, remove clashing values from the neighbours' domains. Empty domain → backtrack now.
- **Arc consistency (AC-3):** also check pairs of unassigned neighbours. After Khulna = Green, Rajshahi {B} and Mymensingh {B} clash → detected even earlier.

Choosing the Order: CSP Heuristics

Heuristic	Rule	In our map
Minimum Remaining Values (MRV)	Pick the variable with the FEWEST legal values left	After Sylhet = Green, pick Mymensingh (only Blue left)
Degree heuristic	Break ties: pick the variable with the MOST constraints	Start with Dhaka (6 neighbours)
Least Constraining Value	Try the value that rules out the FEWEST options for neighbours	Prefer a colour that neighbours do not need

- With MRV, the dead end in our example is avoided completely — try it as homework.

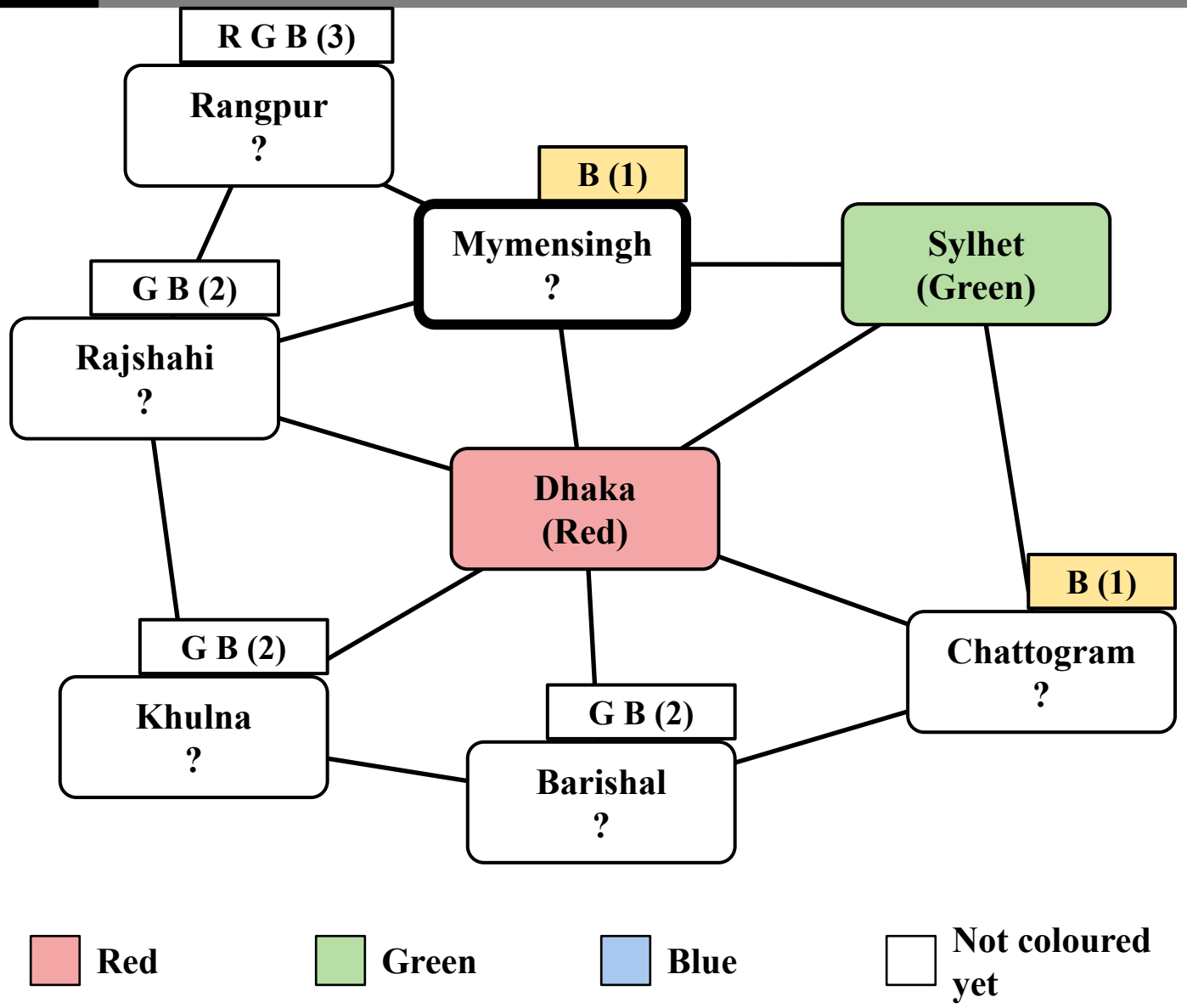
Degree Heuristic: Which Variable First?



Rule

- Pick the variable with the <u>most constraints</u> on other <u>unassigned</u> variables.
- Here: <u>Dhaka</u> (6 neighbours) → colour it first.
- <u>Where it helps:</u>
  - At the <u>start</u>, when all domains are the same size
  - As a <u>tie-breaker</u> for MRV
  - Colouring a busy variable early <u>shrinks many other domains</u> → smaller search tree

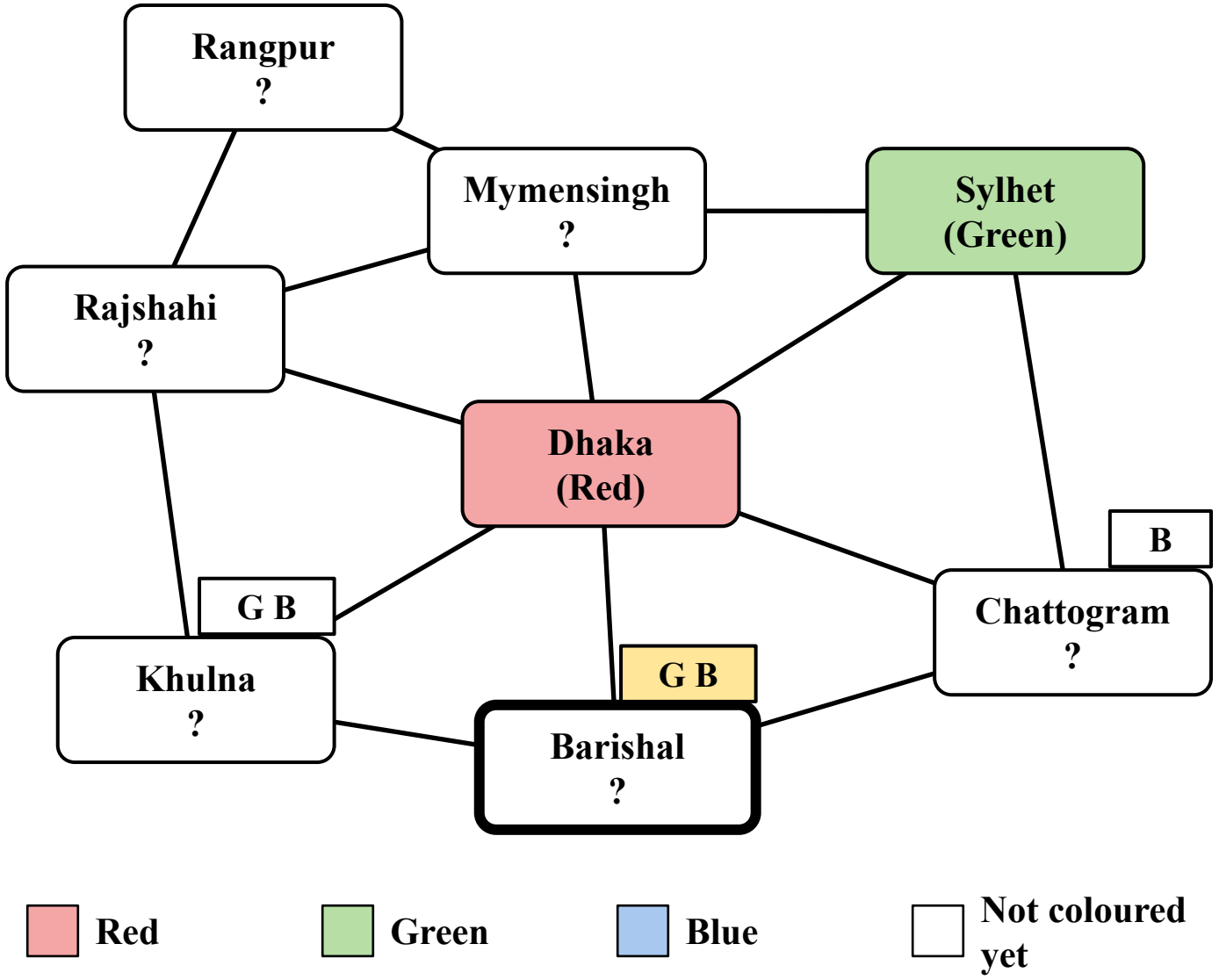
MRV: Minimum Remaining Values



Rule

- Pick the variable with the <u>fewest legal values</u> left.
- Here: Mymensingh and Chattogram have <u>only Blue</u> → pick one now (tie → degree → <u>Mymensingh</u>).
- <u>Where it helps:</u>
  - "Fail first": a variable that will fail is found <u>early</u>, before wasted work
  - Works best together with <u>forward checking</u>
  - With MRV our map needs <u>0 backtracks</u> (fixed order needed 1)

LCV: Least Constraining Value



Colouring Barishal

- Try each colour and count what the neighbours keep:

Barishal =	Khulna	Chattogram	Choices left
Green	{B}	{B}	2 ✓
Blue	{G}	{ } ✗	1

- LCV tries Green first: it rules out the fewest options for neighbours.
- Where it helps:
  - When we need one solution: leaves the most room to succeed
  - Avoids choices that cause dead ends later
  - Not useful if we need all solutions (every value is tried anyway)

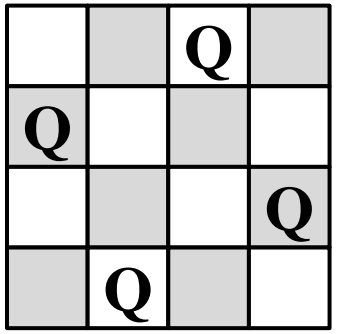
CSP Heuristics: When to Use Which

Heuristic	Chooses	Idea	Best used when…
MRV	Next variable	Fail first: fewest values left	Some variables are much more constrained than others
Degree	Next variable (tie-break)	Most constraints with unassigned variables	At the start, or when MRV gives a tie
LCV	Order of values	Succeed first: leave neighbours the most options	We need just one solution

- Typical solver: MRV + degree to pick the variable, LCV to order its values, forward checking after each assignment.

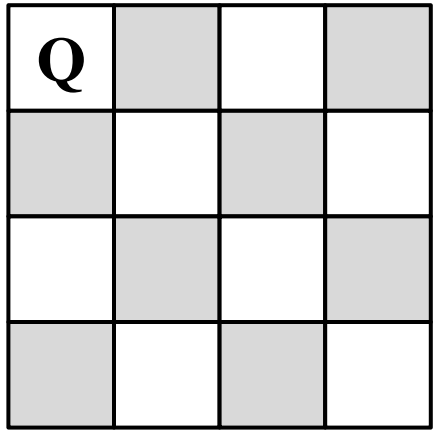
N-Queens as a CSP

- **Place N queens so that no two attack each other.**
- **Variables: Q1 … Q4 (the queen in each column)**
- **Domain: rows {1, 2, 3, 4}**
- **Constraints: no two queens in the same row or diagonal (columns differ by design)**
- **Backtracking: place queens column by column; backtrack when a column has no safe row.**

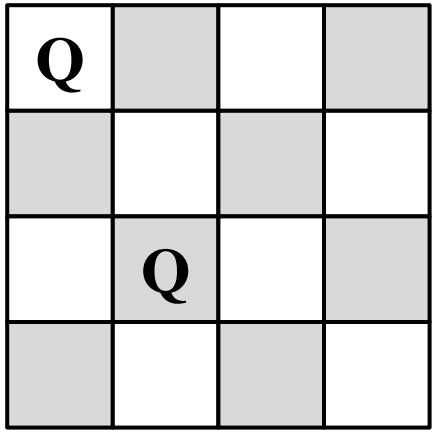


A solution: rows (2, 4, 1, 3)

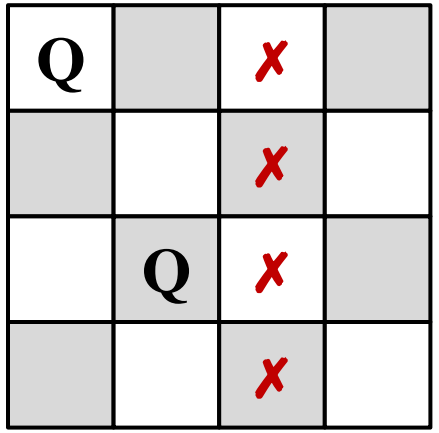
4-Queens Backtracking: Step by Step (1/2)



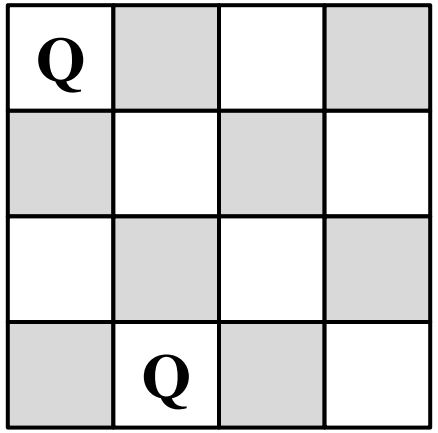
1. Q1 → row 1



2. Q2 → row 3 (rows 1, 2 attacked)



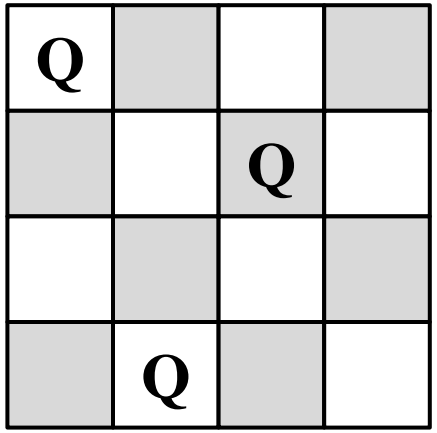
3. Column 3: every row attacked → backtrack



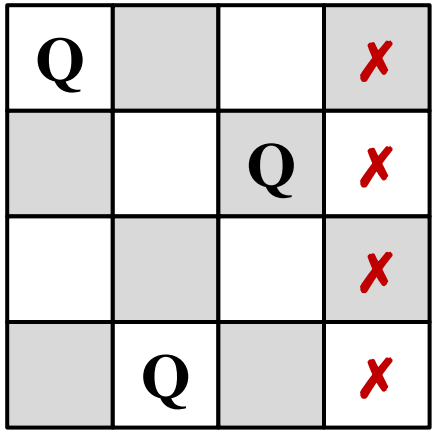
4. Move Q2 → row 4

Row 1 = top row. Q = queen, ✗ = attacked square.

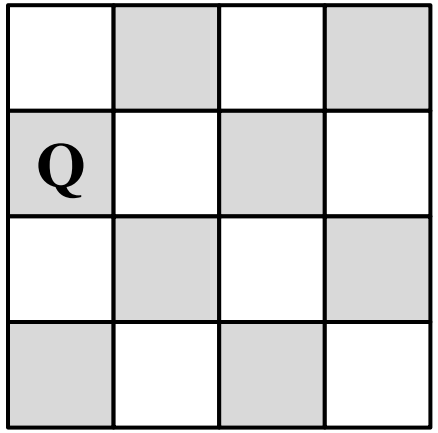
4-Queens Backtracking: Step by Step (2/2)



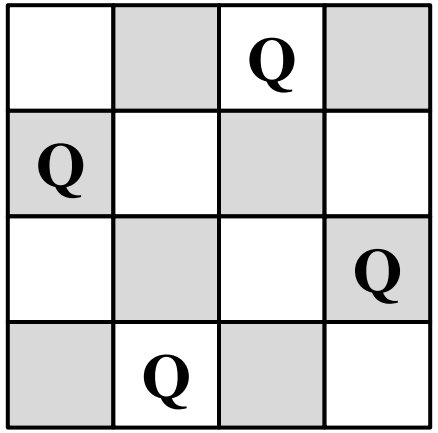
5. Q3 → row 2



6. Column 4: dead end → backtrack to Q1



7. Q1 → row 2



8. Q2 = 4, Q3 = 1, Q4 = 3 → SOLVED ✓

Row 1 = top row. Q = queen, ✗ = attacked square.

Map Colouring in Python

```
neighbours = {
  'Dhaka':      ['Rajshahi','Mymensingh','Sylhet','Chattogram','Khulna','Barishal'],
  'Sylhet':     ['Mymensingh','Dhaka','Chattogram'],
  'Khulna':     ['Rajshahi','Dhaka','Barishal'],
  'Rajshahi':   ['Rangpur','Mymensingh','Dhaka','Khulna'],
  'Rangpur':    ['Rajshahi','Mymensingh'],
  'Mymensingh': ['Rangpur','Rajshahi','Dhaka','Sylhet'],
  'Barishal':   ['Khulna','Dhaka','Chattogram'],
  'Chattogram': ['Sylhet','Dhaka','Barishal'] }
order = ['Dhaka','Sylhet','Khulna','Rajshahi','Mymensingh','Chattogram','Barishal','Rangpur']

def backtrack(assign, i=0):
    if i == len(order): return assign
    var = order[i]
    for c in ['Red', 'Green', 'Blue']:
        if all(assign.get(n) != c for n in neighbours[var]):
            assign[var] = c
            if backtrack(assign, i + 1): return assign
            del assign[var]          # backtrack
    return None

print(backtrack({}))
```

Class Exercise (5 minutes)

- **A class routine has 4 periods (P1–P4) and 3 teachers (A, B, C).**
- **Constraints:**
  - **The same teacher cannot take two periods in a row.**
  - **Only teacher C can take P4 (C is busy in P1–P3).**
- **1. Write the variables, domains and constraints.**
- **2. Find a valid routine with backtracking.**
- **3. Which variable would the MRV heuristic choose first? Why?**

Quick Check

- **1. Which approach to AI does this course follow?**
- **2. What is the difference between a recoverable and an irrecoverable problem? Give an example of each.**
- **3. What is a belief state?**
- **4. What are the three parts of a CSP?**
- **5. How does forward checking improve backtracking?**

Summary

- AI can be defined by thinking/acting × humanly/rationally; we use the rational agent.
- Problems are defined as state-space search (water jug) and judged by 7 problem characteristics.
- With partial information, agents search over belief states or build contingency plans.
- A CSP = variables + domains + constraints; a solution satisfies every constraint.
- Backtracking assigns one variable at a time and undoes choices at dead ends; forward checking, arc consistency and MRV make it faster.
- Reading: Russell & Norvig, Ch. 1, 3.6 (2nd ed.) and Ch. 6; Rich & Knight, Ch. 1–2.
- Next week: Game Playing — Minimax and Alpha–Beta

Thank You

Any questions?